

```

import os, warnings, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             roc_curve, roc_auc_score, RocCurveDisplay)

warnings.filterwarnings("ignore")
RNG = 42
np.random.seed(RNG)

FIGDIR = "figures"
os.makedirs(FIGDIR, exist_ok=True)

plt.rcParams.update({"figure.dpi": 110, "font.size": 10, "axes.grid": True,
                    "grid.alpha": 0.3, "figure.autolayout": True})
print("Environment ready.")

```

Environment ready.

✓ 1. Data acquisition and feature engineering

We download ~7 years of daily SPY prices and engineer a compact set of standard technical features. The prediction target is the **sign of the next day's return**.

```

# --- Load prices (live, with a self-contained fallback) -----
TICKER, START, END = "SPY", "2015-01-01", "2024-12-31"

def load_prices():
    try:
        import yfinance as yf
        df = yf.download(TICKER, start=START, end=END, auto_adjust=True, progress=False)
        if isinstance(df.columns, pd.MultiIndex):
            df.columns = df.columns.get_level_values(0)
        px = df[["Close", "Volume"]].dropna()
        if len(px) < 500:
            raise ValueError("insufficient rows")
        print(f"Loaded {len(px)} live daily bars for {TICKER}.")
        return px
    except Exception as e:
        print(f"[fallback] live download unavailable ({e}); simulating GBM prices.")
        n = 2400
        mu, sigma = 0.08/252, 0.16/np.sqrt(252)
        rets = np.random.normal(mu, sigma, n)
        close = 100*np.exp(np.cumsum(rets))
        vol = np.random.lognormal(15.5, 0.4, n)
        idx = pd.bdate_range("2015-01-01", periods=n)
        return pd.DataFrame({"Close": close, "Volume": vol}, index=idx)

px = load_prices()
px.head()

```

Loaded 2515 live daily bars for SPY.

	Price	Close	Volume
Date			
2015-01-02	169.687790	121465900	
2015-01-05	166.623367	169632600	
2015-01-06	165.053909	209151400	
2015-01-07	167.110687	125346700	
2015-01-08	170.076035	147217800	

```

# --- Feature engineering -----
def rsi(series, window=14):
    delta = series.diff()
    gain = delta.clip(lower=0).rolling(window).mean()
    loss = (-delta.clip(upper=0)).rolling(window).mean()

```

```

rs = gain / (loss.replace(0, np.nan))
return 100 - 100/(1+rs)

df = pd.DataFrame(index=px.index)
ret1 = px["Close"].pct_change()
df["ret1"] = ret1 # 1-day return
df["ret5"] = px["Close"].pct_change(5) # 1-week return
df["mom10"] = px["Close"].pct_change(10) # 10-day momentum
df["vol10"] = ret1.rolling(10).std() # 10-day realised vol
df["ma_ratio"] = px["Close"]/px["Close"].rolling(20).mean() - 1 # dist. from 20d MA
df["rsi14"] = rsi(px["Close"], 14) # relative strength index
df["vol_chg"] = px["Volume"].pct_change().clip(-3, 3) # volume change

# Target: does the NEXT day close up?
df["target"] = (ret1.shift(-1) > 0).astype(int)
df = df.dropna()

FEATURES = ["ret1", "ret5", "mom10", "vol10", "ma_ratio", "rsi14", "vol_chg"]
print("Feature matrix shape:", df[FEATURES].shape)
print("\nClass balance (target):")
print(df['target'].value_counts(normalize=True).round(3).rename({0: 'down', 1: 'up'}))
df[FEATURES + ["target"]].head()

```

Feature matrix shape: (2496, 7)

Class balance (target):

up 0.547

down 0.453

Name: target, dtype: float64

	ret1	ret5	mom10	vol10	ma_ratio	rsi14	vol_chg	target
Date								
2015-01-30	-0.012575	-0.026931	0.002161	0.010750	-0.015463	39.839125	0.139092	1
2015-02-02	0.012384	-0.017182	0.001438	0.010655	-0.002406	48.509649	-0.175101	1
2015-02-03	0.014461	0.010358	0.013758	0.011577	0.011241	55.141461	-0.238458	0
2015-02-04	-0.003807	0.019586	0.004826	0.011608	0.006337	56.060731	0.081262	1
2015-02-05	0.010095	0.020447	0.000097	0.011037	0.015542	63.336028	-0.270675	0

```

# --- Time-ordered split + standardisation -----
X = df[FEATURES].values
y = df["target"].values

```

```

split = int(len(X)*0.75) # chronological split (no shuffle)
X_train_raw, X_test_raw = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

```

```

scaler = StandardScaler().fit(X_train_raw) # fit on training data only
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

```

```

print(f"Train: {X_train.shape[0]} rows | Test: {X_test.shape[0]} rows")
print(f"Train dates: {df.index[0].date()} -> {df.index[split-1].date()}")
print(f"Test dates: {df.index[split].date()} -> {df.index[-1].date()}")

```

```

Train: 1872 rows | Test: 624 rows
Train dates: 2015-01-30 -> 2022-07-07
Test dates: 2022-07-08 -> 2024-12-30

```

✧ Question 5 — Linear Discriminant Analysis (Category 5)

LDA is a **generative linear classifier**. It models each class as a Gaussian with a *shared* covariance matrix and assigns each observation to the class with the highest posterior probability, which reduces to a set of **linear discriminant functions**.

```

# Q5.1 - Fit LDA and score on the held-out test set -----
lda = LinearDiscriminantAnalysis(solver="svd")
lda.fit(X_train, y_train)

y_pred_lda = lda.predict(X_test)
y_proba_lda = lda.predict_proba(X_test)[: , 1]

acc_lda = accuracy_score(y_test, y_pred_lda)
auc_lda = roc_auc_score(y_test, y_proba_lda)

print(f"LDA test accuracy : {acc_lda:.4f}")
print(f"LDA test ROC-AUC : {auc_lda:.4f}\n")

```

```
print("Classification report (0=down, 1=up):")
print(classification_report(y_test, y_pred_lda, target_names=["down", "up"]))
```

```
LDA test accuracy : 0.5353
LDA test ROC-AUC : 0.4664
```

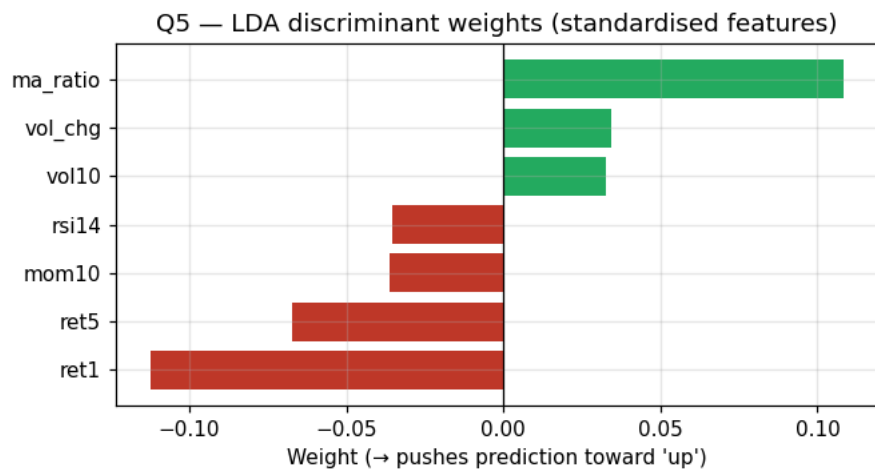
```
Classification report (0=down, 1=up):
```

	precision	recall	f1-score	support
down	0.30	0.01	0.02	286
up	0.54	0.98	0.70	338
accuracy			0.54	624
macro avg	0.42	0.49	0.36	624
weighted avg	0.43	0.54	0.39	624

```
# Q5.2 – Estimated linear discriminant coefficients -----
coef = pd.Series(lda.coef_[0], index=FEATURES).sort_values()
print("LDA discriminant coefficients (standardised features):")
print(coef.round(4))
```

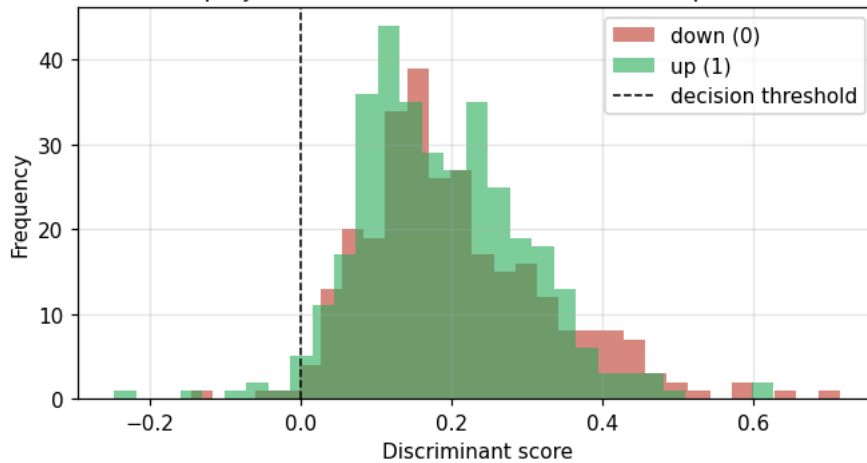
```
fig, ax = plt.subplots(figsize=(6.2, 3.4))
colors = ["#c0392b" if v < 0 else "#27ae60" for v in coef.values]
ax.barh(coef.index, coef.values, color=colors)
ax.axvline(0, color="k", lw=0.8)
ax.set_title("Q5 – LDA discriminant weights (standardised features)")
ax.set_xlabel("Weight (→ pushes prediction toward 'up')")
fig.savefig(f"{FIGDIR}/q5_coefficients.png", bbox_inches="tight")
plt.show()
```

```
LDA discriminant coefficients (standardised features):
ret1      -0.1124
ret5      -0.0673
mom10     -0.0364
rsi14     -0.0356
vol10      0.0325
vol_chg    0.0345
ma_ratio   0.1084
dtype: float64
```



```
# Q5.3 – Distribution of the 1-D discriminant score by class -----
scores = lda.decision_function(X_test) # projection onto the discriminant axis
fig, ax = plt.subplots(figsize=(6.2, 3.6))
ax.hist(scores[y_test==0], bins=30, alpha=0.6, label="down (0)", color="#c0392b")
ax.hist(scores[y_test==1], bins=30, alpha=0.6, label="up (1)", color="#27ae60")
ax.axvline(0, color="k", ls="--", lw=1, label="decision threshold")
ax.set_title("Q5 – LDA projects data onto one axis that best separates classes")
ax.set_xlabel("Discriminant score"); ax.set_ylabel("Frequency"); ax.legend()
fig.savefig(f"{FIGDIR}/q5_projection.png", bbox_inches="tight")
plt.show()
```

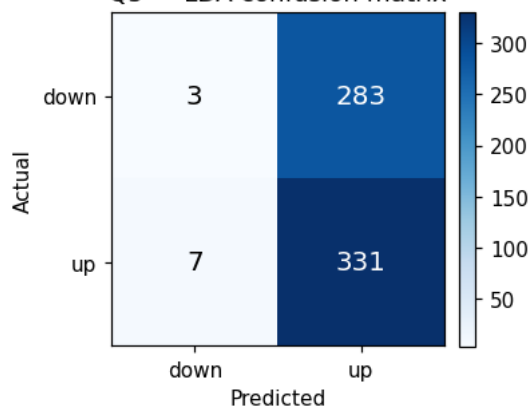
Q5 — LDA projects data onto one axis that best separates classes



```
# Q5.4 — Confusion matrix -----
def plot_cm(cm, title, fname):
    fig, ax = plt.subplots(figsize=(3.8, 3.6))
    im = ax.imshow(cm, cmap="Blues")
    ax.set_xticks([0,1], labels=["down","up"]); ax.set_yticks([0,1], labels=["down","up"])
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual"); ax.set_title(title)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, cm[i,j], ha="center", va="center",
                    color="white" if cm[i,j] > cm.max()/2 else "black", fontsize=13)
    ax.grid(False); fig.colorbar(im, fraction=0.046, pad=0.04)
    fig.savefig(fname, bbox_inches="tight"); plt.show()

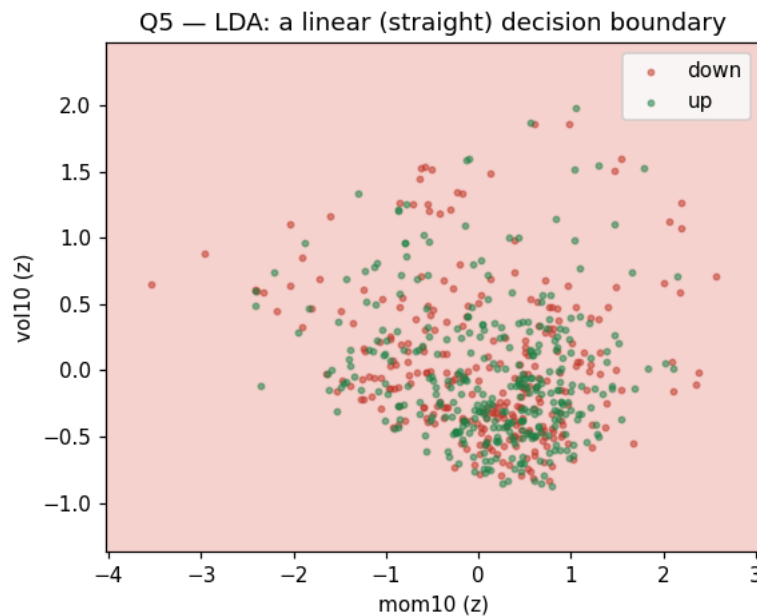
cm_lda = confusion_matrix(y_test, y_pred_lda)
plot_cm(cm_lda, "Q5 — LDA confusion matrix", f"{FIGDIR}/q5_confusion.png")
```

Q5 — LDA confusion matrix

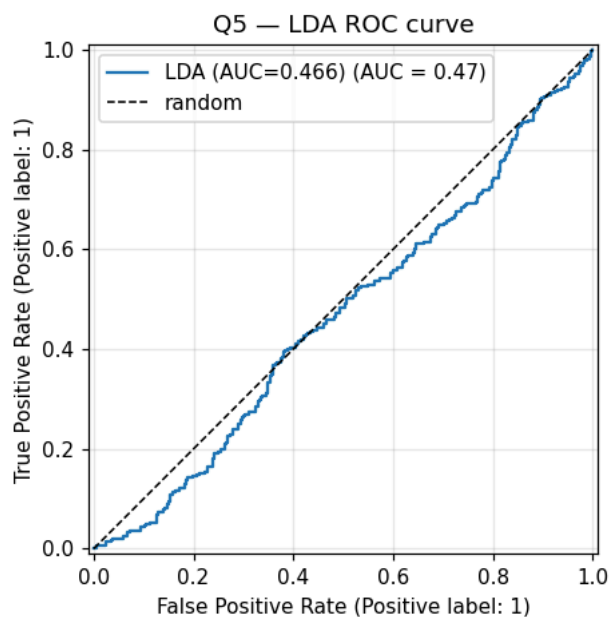


```
# Q5.5 — LDA decision boundary on two features (visualisation) -----
def plot_boundary(model2d, X2, y2, title, fname, feat_names):
    h = .02
    x_min, x_max = X2[:,0].min()-.5, X2[:,0].max()+.5
    y_min, y_max = X2[:,1].min()-.5, X2[:,1].max()+.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = model2d.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    fig, ax = plt.subplots(figsize=(5.4, 4.4))
    ax.contourf(xx, yy, Z, alpha=.25, cmap=ListedColormap(["#e74c3c", "#2ecc71"]))
    ax.scatter(X2[y2==0,0], X2[y2==0,1], s=8, c="#c0392b", label="down", alpha=.5)
    ax.scatter(X2[y2==1,0], X2[y2==1,1], s=8, c="#1e8449", label="up", alpha=.5)
    ax.set_xlabel(feat_names[0]); ax.set_ylabel(feat_names[1])
    ax.set_title(title); ax.legend(); ax.grid(False)
    fig.savefig(fname, bbox_inches="tight"); plt.show()

viz_idx = [FEATURES.index("mom10"), FEATURES.index("vol10")]
X2_train, X2_test = X_train[:, viz_idx], X_test[:, viz_idx]
lda2d = LinearDiscriminantAnalysis().fit(X2_train, y_train)
plot_boundary(lda2d, X2_test, y_test,
              "Q5 — LDA: a linear (straight) decision boundary",
              f"{FIGDIR}/q5_boundary.png", ["mom10 (z)", "vol10 (z)"])
```



```
# Q5.6 – ROC curve -----
fig, ax = plt.subplots(figsize=(5.0, 4.4))
RocCurveDisplay.from_predictions(y_test, y_proba_lda, name=f"LDA (AUC={auc_lda:.3f})", ax=ax)
ax.plot([0,1],[0,1], "k--", lw=1, label="random")
ax.set_title("Q5 – LDA ROC curve"); ax.legend()
fig.savefig(f"{FIGDIR}/q5_roc.png", bbox_inches="tight")
plt.show()
```



Q5 interpretation. LDA fits in closed form (no iterative optimisation) and produces a single straight-line decision boundary. The discriminant weights are directly readable as the marginal contribution of each standardised feature to the "up" vote, giving an interpretable, low-variance baseline signal. Because the boundary is linear, LDA cannot capture interaction/curvature effects — motivating the neural network in Question 7.

✧ Question 7 — Neural Networks (Category 7)

We use a **multi-layer perceptron (MLP)** — a feed-forward neural network with one or more hidden layers of non-linear (ReLU) units, trained by back-propagation. Unlike LDA it can learn curved, interacting decision boundaries, at the cost of more hyperparameters and training effort.

```
# Q7.1 – Hyperparameter tuning with time-series cross-validation -----
param_grid = {
    "hidden_layer_sizes": [(16,), (32,), (32, 16)],
    "alpha": [1e-4, 1e-3, 1e-2],          # L2 regularisation
```

```

    "learning_rate_init": [0.001, 0.01],
}
base_mlp = MLPClassifier(activation="relu", solver="adam", max_iter=600,
                        early_stopping=True, n_iter_no_change=15, random_state=RNG)

tscv = TimeSeriesSplit(n_splits=4)
grid = GridSearchCV(base_mlp, param_grid, cv=tscv, scoring="roc_auc", n_jobs=-1)
grid.fit(X_train, y_train)

print("Best hyperparameters:")
for k, v in grid.best_params_.items():
    print(f"    {k:22s}: {v}")
print(f"\nBest CV ROC-AUC: {grid.best_score_:.4f}")

```

```

Best hyperparameters:
  alpha          : 0.0001
  hidden_layer_sizes : (16,)
  learning_rate_init : 0.01

```

```
Best CV ROC-AUC: 0.5127
```

```

# Q7.2 – Evaluate the tuned network on the test set -----
mlp = grid.best_estimator_
y_pred_mlp = mlp.predict(X_test)
y_proba_mlp = mlp.predict_proba(X_test)[:, 1]

acc_mlp = accuracy_score(y_test, y_pred_mlp)
auc_mlp = roc_auc_score(y_test, y_proba_mlp)

print(f"MLP test accuracy : {acc_mlp:.4f}")
print(f"MLP test ROC-AUC : {auc_mlp:.4f}\n")
print("Classification report (0=down, 1=up):")
print(classification_report(y_test, y_pred_mlp, target_names=["down", "up"]))

```

```

MLP test accuracy : 0.5160
MLP test ROC-AUC : 0.4748

```

```

Classification report (0=down, 1=up):

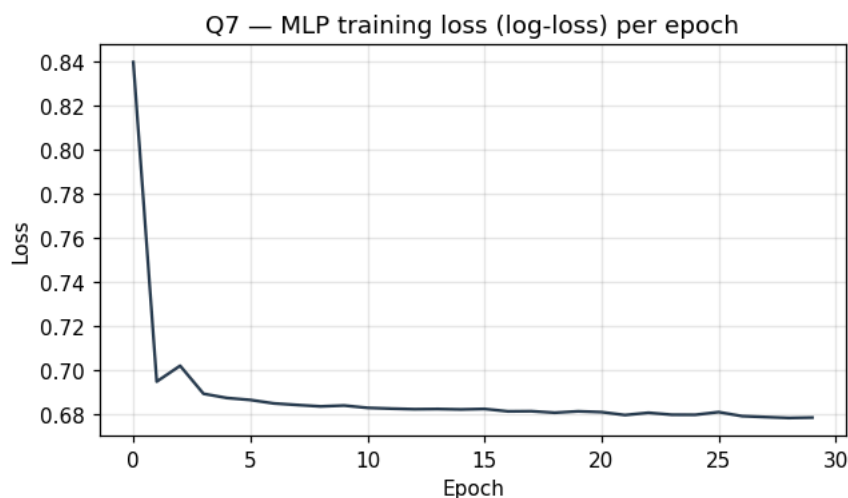
```

	precision	recall	f1-score	support
down	0.43	0.18	0.26	286
up	0.54	0.80	0.64	338
accuracy			0.52	624
macro avg	0.48	0.49	0.45	624
weighted avg	0.49	0.52	0.46	624

```

# Q7.3 – Training loss curve -----
fig, ax = plt.subplots(figsize=(6.0, 3.6))
ax.plot(mlp.loss_curve_, color="#2c3e50")
ax.set_title("Q7 – MLP training loss (log-loss) per epoch")
ax.set_xlabel("Epoch"); ax.set_ylabel("Loss")
fig.savefig(f"{FIGDIR}/q7_loss.png", bbox_inches="tight")
plt.show()

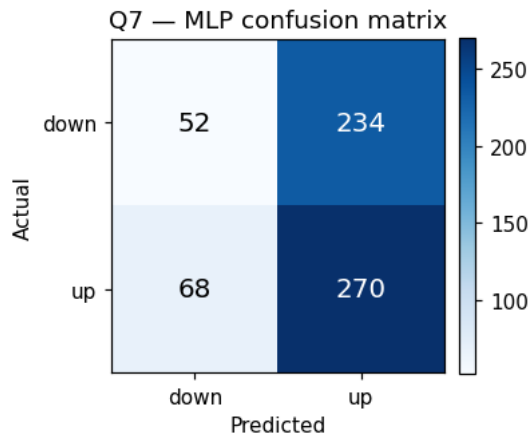
```



```

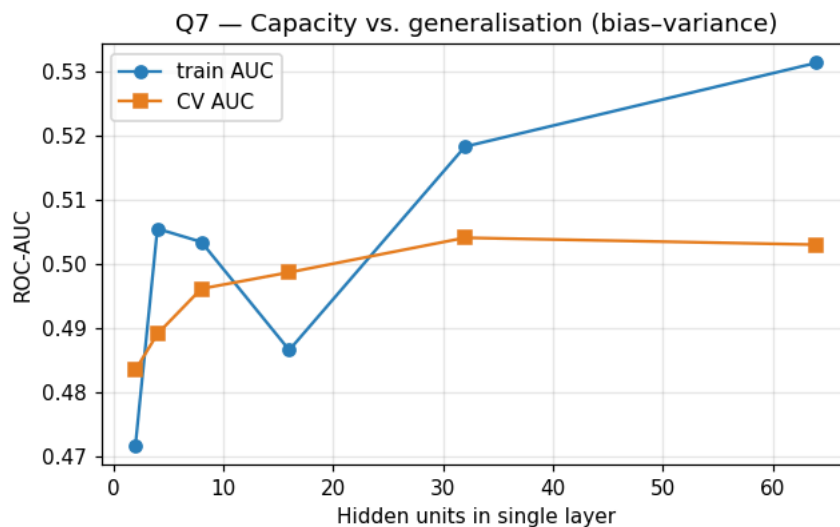
# Q7.4 – Confusion matrix -----
cm_mlp = confusion_matrix(y_test, y_pred_mlp)
plot_cm(cm_mlp, "Q7 – MLP confusion matrix", f"{FIGDIR}/q7_confusion.png")

```

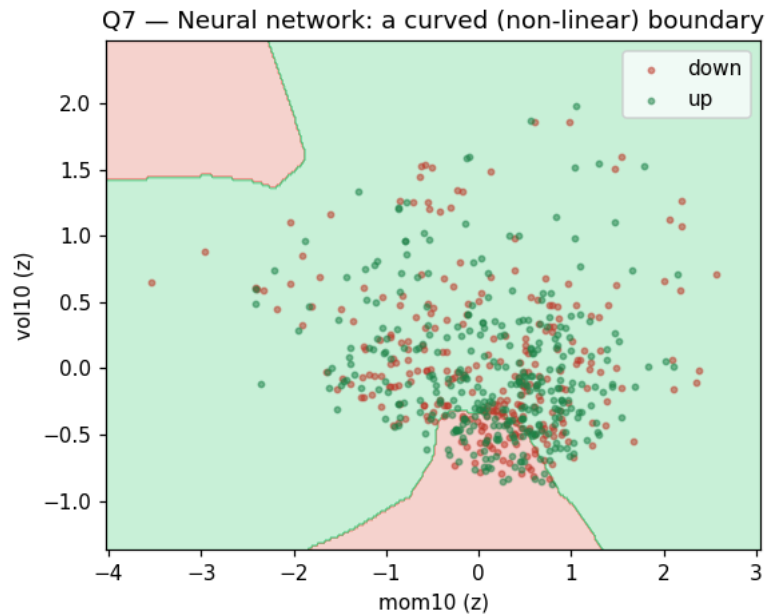


```
# Q7.5 — Validation curve over hidden-layer width -----
from sklearn.model_selection import validation_curve
widths = [2, 4, 8, 16, 32, 64]
train_sc, val_sc = validation_curve(
    MLPClassifier(activation="relu", solver="adam", max_iter=500,
                  alpha=grid.best_params_["alpha"], early_stopping=True, random_state=RNG),
    X_train, y_train, param_name="hidden_layer_sizes",
    param_range=[(w,) for w in widths], cv=tscv, scoring="roc_auc", n_jobs=-1)

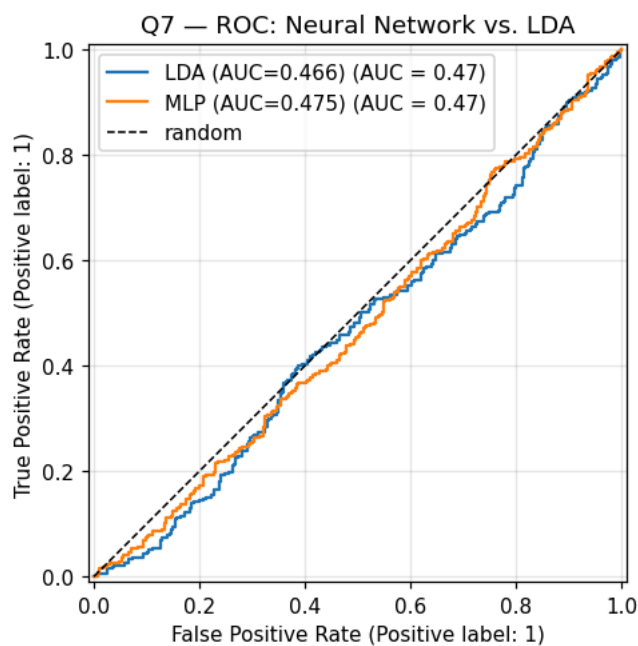
fig, ax = plt.subplots(figsize=(6.0, 3.8))
ax.plot(widths, train_sc.mean(1), "o-", label="train AUC", color="#2980b9")
ax.plot(widths, val_sc.mean(1), "s-", label="CV AUC", color="#e67e22")
ax.set_xlabel("Hidden units in single layer"); ax.set_ylabel("ROC-AUC")
ax.set_title("Q7 — Capacity vs. generalisation (bias-variance)")
ax.legend()
fig.savefig(f"{FIGDIR}/q7_valcurve.png", bbox_inches="tight")
plt.show()
```



```
# Q7.6 — MLP non-linear decision boundary on the same two features -----
mlp2d = MLPClassifier(hidden_layer_sizes=(32, 16), activation="relu", solver="adam",
                       alpha=grid.best_params_["alpha"], max_iter=800,
                       early_stopping=True, random_state=RNG).fit(X2_train, y_train)
plot_boundary(mlp2d, X2_test, y_test,
              "Q7 — Neural network: a curved (non-linear) boundary",
              f"{FIGDIR}/q7_boundary.png", ["mom10 (z)", "vol10 (z)"])
```



```
# Q7.7 — Head-to-head ROC comparison -----
fig, ax = plt.subplots(figsize=(5.2, 4.6))
RocCurveDisplay.from_predictions(y_test, y_proba_lda, name=f"LDA (AUC={auc_lda:.3f})", ax=ax)
RocCurveDisplay.from_predictions(y_test, y_proba_mlp, name=f"MLP (AUC={auc_mlp:.3f})", ax=ax)
ax.plot([0,1],[0,1], "k--", lw=1, label="random")
ax.set_title("Q7 — ROC: Neural Network vs. LDA"); ax.legend()
fig.savefig(f"{FIGDIR}/q7_roc_compare.png", bbox_inches="tight")
plt.show()
```



```
# Q7.8 — Summary comparison table + bar chart -----
summary = pd.DataFrame({
    "Model": ["LDA (Cat 5)", "Neural Net / MLP (Cat 7)"],
    "Accuracy": [acc_lda, acc_mlp],
    "ROC-AUC": [auc_lda, auc_mlp],
}).set_index("Model").round(4)
print(summary)

fig, ax = plt.subplots(figsize=(6.0, 3.4))
summary.plot(kind="bar", ax=ax, color=["#34495e", "#e67e22"], rot=0)
ax.set_ylim(0, 1); ax.set_title("Q5 vs Q7 — Test-set performance")
ax.set_ylabel("Score"); ax.legend(loc="lower right")
for c in ax.containers:
    ax.bar_label(c, fmt="%0.3f", fontsize=8)
fig.savefig(f"{FIGDIR}/comparison.png", bbox_inches="tight")
plt.show()
```

Model	Accuracy	ROC-AUC
LDA (Cat 5)	0.5353	0.4664
Neural Net / MLP (Cat 7)	0.5160	0.4748

